



МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования

«МИРЭА – Российский технологический университет»

РТУ МИРЭА

Институт Информационных технологий

Кафедра информационных технологий в атомной энергетике

Практическая работа 1

по дисциплине «Информационные системы управления данными и документами»

Студент группы ИКБО-50-23

Враженко Д. О.

(подпись студента)

Принял преподаватель

Тесла-Блозва С. А.

(подпись руководителя)

Работа представлена

« ____ » _____ 2026 г.

Допущен к работе

« ____ » _____ 2026 г.

Москва 2026

Оглавление

1. Цель работы	3
2. Ход выполнения работы	3
2.1 Часть 1. Подготовка хост-системы.....	3
2.2 Часть 2. Конфигурация окружения	3
2.3 Часть 3. Запуск и проверка компонентов	4
2.4 Тестирование соединения (Smoke-test)	5
3. Вывод.....	6
4. Ответы на контрольные вопросы	7

1. Цель работы

Подготовить рабочее окружение, развернуть демонстрационный стенд платформы с использованием Docker и Docker Compose, проверить корректность взаимодействия между компонентами (PostgreSQL и вспомогательными сервисами).

2. Ход выполнения работы

2.1 Часть 1. Подготовка хост-системы

Проверка установленного Docker и Docker-Compose представлена на Рисунке 1.

```
PS C:\Users\wrada> docker --version
Docker version 29.7.2, build a7dcaa6
PS C:\Users\wrada> docker-compose --version
Docker Compose version v5.5.0
```

Рисунок 1 – Результат вывода команд

Проверка доступности порта 5432 (чтобы избежать конфликта с локальным PostgreSQL) представлена на Рисунке 2.

```
PS C:\Users\wrada> netstat -ano | findstr :5432
UDP    0.0.0.0:54320      *:*              9272
UDP    0.0.0.0:54322      *:*              9272
UDP    0.0.0.0:54323      *:*              9272
UDP    0.0.0.0:54324      *:*              9272
UDP    0.0.0.0:54325      *:*              9272
UDP    0.0.0.0:54327      *:*              9272
UDP    0.0.0.0:54328      *:*              9272
```

Рисунок 2 – Результат проверки доступности порта 5432

2.2 Часть 2. Конфигурация окружения

Создана рабочая директория проекта представлена на Рисунке 3.

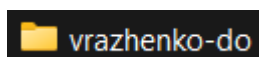
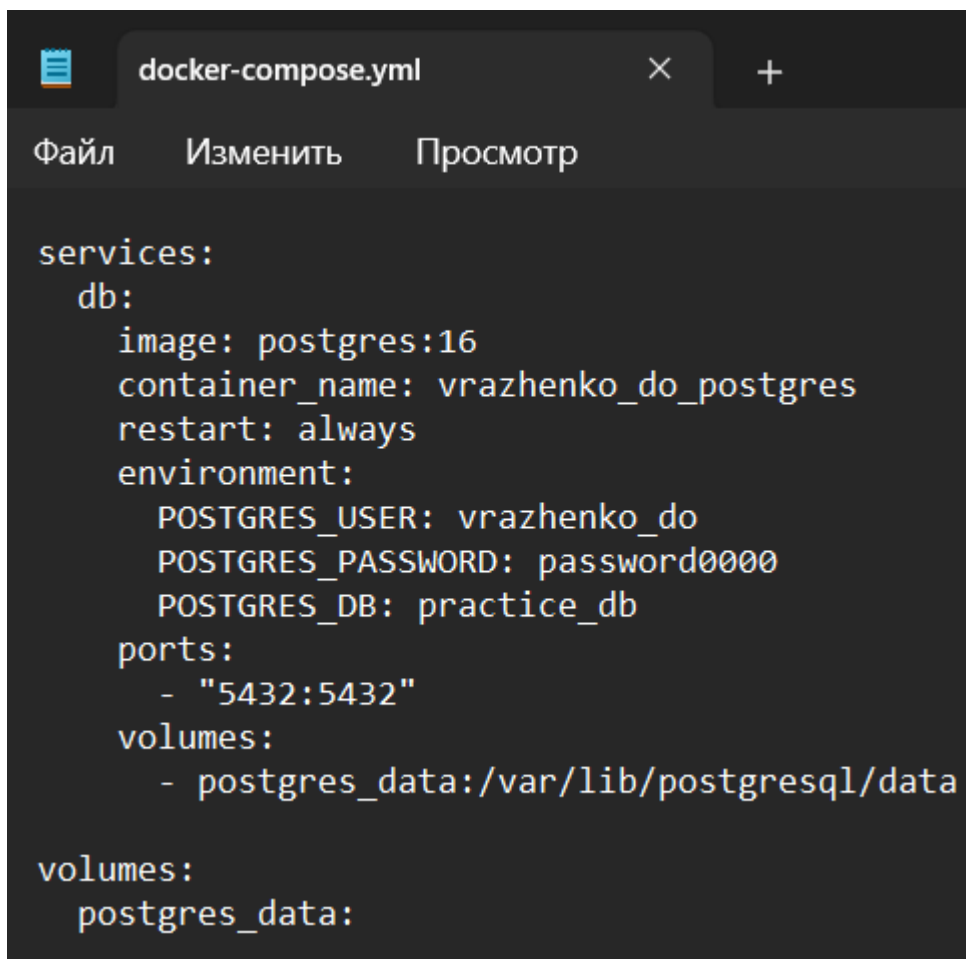


Рисунок 3 – Результат создания рабочей директории проекта

Подготовлен файл `docker-compose.yml`, представленный на Рисунке 4.



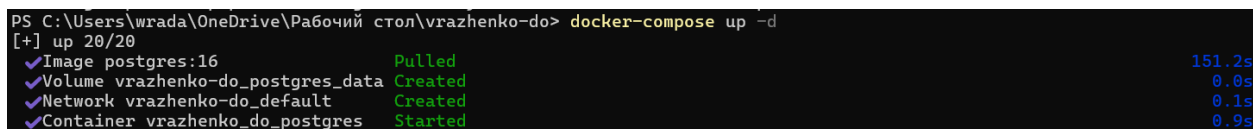
```
services:
  db:
    image: postgres:16
    container_name: vrazhenko_do_postgres
    restart: always
    environment:
      POSTGRES_USER: vrazhenko_do
      POSTGRES_PASSWORD: password0000
      POSTGRES_DB: practice_db
    ports:
      - "5432:5432"
    volumes:
      - postgres_data:/var/lib/postgresql/data

volumes:
  postgres_data:
```

Рисунок 4 – Подготовленный файл `docker-compose.yml`

2.3 Часть 3. Запуск и проверка компонентов

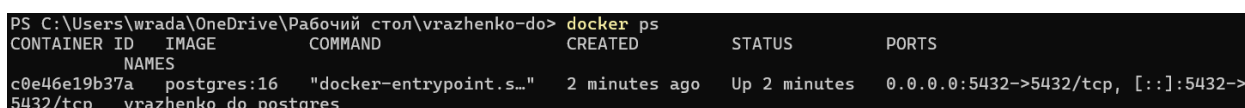
Запуск контейнеров в фоновом режиме представлен на Рисунке 5.



```
PS C:\Users\wrada\OneDrive\Рабочий стол\vrazhenko-do> docker-compose up -d
[+] up 20/20
✓Image postgres:16 Pulled 151.2s
✓Volume vrazhenko-do_postgres_data Created 0.0s
✓Network vrazhenko-do_default Created 0.1s
✓Container vrazhenko-do_postgres Started 0.9s
```

Рисунок 5 – Результат запуска контейнера в фоновом режиме

Проверка статуса контейнера представлена на Рисунке 6.



CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS
c0e46e19b37a	postgres:16	"docker-entrypoint.s..."	2 minutes ago	Up 2 minutes	0.0.0.0:5432->5432/tcp, [::]:5432->5432/tcp
vrazhenko_do_postgres					

Рисунок 6 – Результат проверки статуса контейнера

Просмотр логов базы данных (убедиться, что инициализация прошла успешно) представлен на Рисунке 7.

```
PS C:\Users\wrada\OneDrive\Рабочий стол\vrazhenko-do> docker logs vrazhenko_do_postgres
The files belonging to this database system will be owned by user "postgres".
This user must also own the server process.

The database cluster will be initialized with locale "en_US.utf8".
The default database encoding has accordingly been set to "UTF8".
The default text search configuration will be set to "english".

Data page checksums are disabled.

fixing permissions on existing directory /var/lib/postgresql/data ... ok
creating subdirectories ... ok
selecting dynamic shared memory implementation ... posix
selecting default max_connections ... 100
selecting default shared_buffers ... 128MB
selecting default time zone ... Etc/UTC
creating configuration files ... ok
running bootstrap script ... ok
performing post-bootstrap initialization ... ok
syncing data to disk ... ok

Success. You can now start the database server using:

    pg_ctl -D /var/lib/postgresql/data -l logfile start
initdb: warning: enabling "trust" authentication for local connections
initdb: hint: You can change this by editing pg_hba.conf or using the option -A, or --auth-local and --auth-host, the ne
xt time you run initdb.
waiting for server to start...2026-09-10 10:51:39.907 UTC [48] LOG:  starting PostgreSQL 16.15 (Debian 16.15-1.pgdg13+2
) on x86_64-pc-linux-gnu, compiled by gcc (Debian 14.2.0-19) 14.2.0, 64-bit
2026-09-10 10:51:39.909 UTC [48] LOG:  listening on Unix socket "/var/run/postgresql/.s.PGSQL.5432"
2026-09-10 10:51:39.915 UTC [51] LOG:  database system was shut down at 2026-09-10 10:51:39 UTC
2026-09-10 10:51:39.920 UTC [48] LOG:  database system is ready to accept connections
done
server started
CREATE DATABASE
```

Рисунок 7 – Результат просмотра логов базы данных

2.4 Тестирование соединения (Smoke-test)

Для проверки использовался клиент DBeaver. Параметры подключения представлены на Рисунке 8.

The screenshot shows the DBeaver 'Driver Properties' window for a PostgreSQL connection. The 'Server' tab is active, showing the following configuration:

- Connect by:** Host (selected), URL
- URL:** jdbc:postgresql://localhost:5432/practice_db
- Хост:** localhost
- Порт:** 5432
- База данных:** practice_db
- Аутентификация:** Username/password
- Пользователь:** vrazhenko_do
- Пароль:** [masked]
- Сохранять пароль:** checked

At the bottom, there are links for 'Переменные окружения' (Environment Variables) and 'PostgreSQL', and buttons for 'Настройки драйвера' (Driver Settings) and 'Лицензия драйвера' (Driver License).

Рисунок 8 – Параметры подключения

Параметры подключения:

Хост: localhost

Порт: 5432

База данных: practice_db

Пользователь: vrazhenko_do

Пароль: password0000

Результат успешного подключения представлен на Рисунке 9:

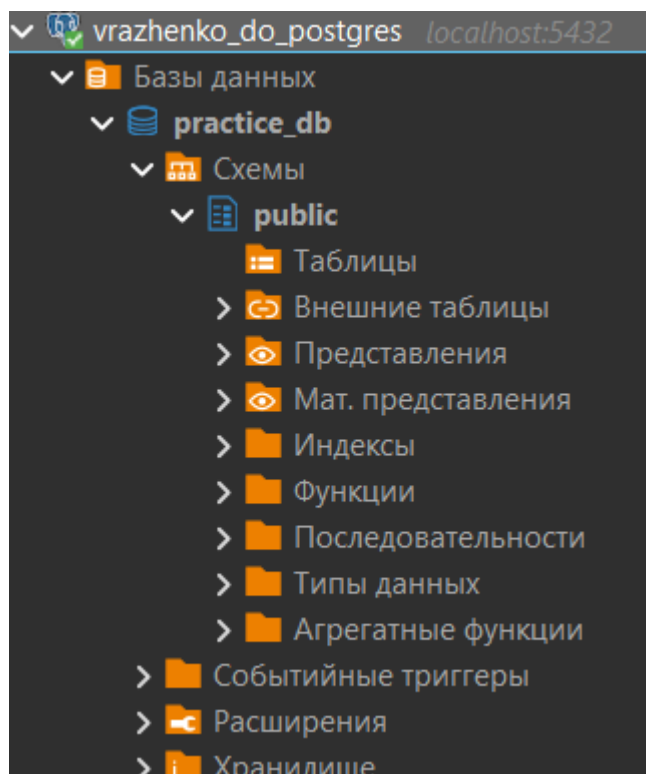


Рисунок 9 – Результат успешного подключения

3. Вывод

В ходе практической работы была развёрнута базовая инфраструктура с использованием Docker Compose. Успешно запущен контейнер PostgreSQL, подтверждена корректная инициализация базы данных и возможность подключения через внешний клиент. Получены навыки работы с docker-compose.yml, управления контейнерами и томами.

4. Ответы на контрольные вопросы

1. В чем заключается преимущество запуска PostgreSQL в контейнере по сравнению с классической установкой в операционную систему?

- Изоляция: контейнер не конфликтует с другими приложениями и системными библиотеками.
- Портруемость: один и тот же образ работает на любой машине с Docker — не нужно настраивать зависимости вручную.
- Быстрота развёртывания: контейнер запускается за секунды, тогда как классическая установка требует длительной настройки.
- Лёгкость управления версиями: можно легко переключаться между версиями PostgreSQL, просто меняя тег образа.
- Воспроизводимость: конфигурация описана в `docker-compose.yml`, что исключает ошибки «человеческого фактора».

2. Какая команда позволяет остановить все сервисы и одновременно удалить созданные контейнеры и сети?

```
docker-compose down
```

Эта команда останавливает и удаляет контейнеры, а также сети, созданные при `docker-compose up`. Важно: по умолчанию тома (`volumes`) не удаляются.

3. Что произойдет с данными в базе, если удалить контейнер, но не удалить volume (том), примонтированный к нему?

Данные сохранятся. Тома — это механизм временного хранения, управляемый Docker-демоном. Они сохраняют данные независимо от жизненного цикла контейнера. Если позже запустить новый контейнер с тем же томом, данные будут доступны.

4. Для чего в `docker-compose.yml` используется параметр `restart: always`?

Параметр `restart: always` указывает Docker автоматически перезапускать контейнер в случае:

- сбоя процесса внутри контейнера;
- перезагрузки хост-системы;
- остановки Docker-демона.

Это обеспечивает высокую доступность сервиса без ручного вмешательства.

5. Как зайти внутрь работающего контейнера и выполнить в нем команду (например, `psql`) через терминал?

Используйте команду `docker exec` с флагами `-it` (интерактивный режим с TTY):

```
docker exec -it ivanov_ii_postgres psql -U student -d practice_db
```

Для запуска оболочки `bash` внутри контейнера:

```
docker exec -it ivanov_ii_postgres bash
```

Затем внутри контейнера можно выполнить `psql -U student -d practice_db`.